

Runtime Function Reference

Two-country HANK–GK monetary union — code/global/

Model documentation

2026-07-14

Contents

1. Execution overview	2
2. Notation and conventions	3
3. Module transition	3
_inner_economy	4
solve_transition	5
residual (nested in solve_transition)	6
solve_transition_ck	7
4. Module risk_branch	7
extract_init_state	8
solve_default_branch	8
make_risk_inputs	9
solve_transition_ck_risk	9
bond_decomposition	10
5. Module household	11
make_asset_grid	11
egm_step	11
solve_steady_state_household	12
solve_backward_transition	12
6. Module firms	12
markup_ss	12
steady_state_firm	13
solve_firm_path	13

7. Module government	13
hm_bond_price_ss / hm_bond_return_ss	13
govt_steady_state	14
ck_default_prob	14
govt_transition	14
8. Module capital	15
gamma_params	15
capital_demand	15
solve_capital_path	16
9. Module distribution	16
get_lottery_weights	17
forward_iterate	17
stationary_distribution	17
aggregate_assets / aggregate_consumption	17
10. Call-graph summary	18

Scope. This document is a practical reference for the principal functions on the main execution path of the two-country HANK–GK monetary-union model (python3 code/global/main.py). It covers seven modules — `transition`, `risk_branch`, `household`, `firms`, `government`, `capital`, `distribution` — and, for each core function, records its interface, internal logic, economic purpose, and computational properties. The bank block (`bank.py`), trade block (`trade.py`), steady-state driver (`steady_state.py`) and plotting are referenced where they interact with these modules but are not documented in detail here.

Status. Reflects the code as of 2026-07-14 (branch `global`). Parameter values cited are the current entries in `calibration.py` and may drift; treat parameter *names* as authoritative and re-check values there.

1. Execution overview

`main.py` runs three sections in order:

1. **Steady state (always runs).** `steady_state.solve_steady_state(cal)` solves the symmetric two-country steady state (two-stage: capital markets + current account, then deposit markets). Every transition experiment starts from and terminates at this steady state.
2. **TFP shock (RUN_TFP).** A 1% AR(1) TFP shock in country D ($Z_D[t] = Z_{ss} \cdot \exp(0.01 \cdot 0.8^t)$) is fed to `solve_transition` with no default risk — the baseline real-shock IRF used to validate the perfect-foresight machinery.
3. **Cole-Kehoe sunspot + Bocola risk channel (RUN_SUNSPOT, centerpiece).** A sunspot path ($\xi[t] = 0.01 \cdot 0.95^t$) raises the *priced* probability of sovereign default while TFP stays flat. `solve_transition_ck_risk` solves the fixed point between the no-default base path and a representative post-default branch; `bond_decomposition` then splits the sovereign spread into default compensation, risk premium, and liquidity premium (plotted as a standalone figure, toggled by `PLOT_BOND_DECOMPOSITION` in `main.py`). Default is priced but never realized (`def_real` $\equiv$ 0): pure pass-through.

The solver hierarchy, from outermost to innermost:

solve_transition_ck_risk	(risk_branch)	base ↔ default-branch fixed point
└ solve_transition_ck	(transition)	crisis-zone indicator fixed point
└└ solve_transition	(transition)	7T-unknown Newton (scipy root/hybr)
└└└ residual	(nested)	one full economy per evaluation
└└└└ _inner_economy	(transition)	firms → capital → banks → govt → households → distribution → trade

Every Newton residual evaluation solves a *complete* general-equilibrium economy given the 7T guessed paths — there are no inner fixed points besides the household backward/forward passes, which are direct (non-iterative) given prices.

2. Notation and conventions

Symbol / suffix	Meaning
D, F	Country suffixes: D = domestic/periphery (Greece), F = foreign/core (Germany).
T	Transition horizon in quarters (<code>cal["T"]</code>).
p	Relative price of the F good in D goods (terms of trade / real exchange rate); $p_{ss} = 1$ in the symmetric steady state.
P_CES	CES consumption-basket price index in units of the home good.
N, Kap	Aggregate employment and end-of-period capital stock.
rdep	Deposit rate set at t, paid at t+1 (predetermined: the rate received at t was locked at t–1).
Q, rk	Price of capital (Jermann) and realized return on capital claims.
Q_bD, Q_bF	Hatchondo-Martinez perpetuity prices for D- and F-government bonds.
def_price, def_real	<i>Priced</i> default probability (enters Q and expected-return FOCs) vs <i>realized</i> haircut indicator (enters realized return FOCs).
b_gov, b_gov_eop	Government bond stock at beginning / end of period.
n, alpha, mu	Bank net worth, franchise value per unit of net worth (V/n), and incentive-constraint multiplier.
vN	GHH labour disutility $\chi \cdot N^{(1+1/\text{frisch})}/(1+1/\text{frisch})$; the GHH composite is $x = c - vN$.
ss, cal	Steady-state dict from <code>solve_steady_state</code> ; calibration dict from <code>get_calibration</code> .

Bond denomination convention (from `calibration.py`): D-bonds are D-good claims priced off $rdep_D$; F-bonds are F-good claims priced off $rdep_F$. Cross-border positions convert at p (e.g. the D-bank's F-bond leg in D-goods is $p \cdot Q_{bF} \cdot b_{F_D}$).

3. Module transition

File: `code/global/transition.py`

The nonlinear perfect-foresight (MIT-shock) transition solver. Stacks 7 unknown paths of length T into a single vector and solves 7T market-clearing residuals with `scipy.optimize.root`. Two design decisions define the module:

- **Endogenous debt inside every residual call.** Bond prices come from bank marginal conditions alone (`bank_backward`); the debt stock is then forward-integrated from the government budget identity (`govt_transition`), and banks clear the bond market against the *true* end-of-period stock. This keeps Walras exact when beliefs move the debt stock — clearing against a fixed B_{gov_ss} re-opens a leak of ~0.5% of GDP per 5% debt deviation.
- **Walras redundancy.** The F goods market and the current account are *dropped* from the residual system and only monitored as diagnostics (thresholds: $goods_D \leq 1e-9$ imposed, $goods_F \leq 2e-6$ diagnostic).

`_inner_economy`

```
_inner_economy(N_D, N_F, Kap_D, Kap_F, rdep_D, rdep_F, p_path,
               Z_D_path, Z_F_path, ss, cal,
               def_price_D=None, def_price_F=None,
               def_real_D=None, def_real_F=None,
               init=None, risk_D=None) -> dict
```

Input	Type	Role
<code>N_*</code> , <code>Kap_*</code> , <code>rdep_*</code> , <code>p_path</code>	(T,) arrays	The 7 guessed unknown paths.
<code>Z_*</code> , <code>Z_path</code>	(T,) arrays	Exogenous TFP paths.
<code>def_price_*</code> , <code>def_real_*</code>	(T,) or None	Priced default probability / realized haircut paths.
<code>init</code>	dict or None	Mid-crisis initial conditions (lagged states) for default branches and policy runs; None $\Rightarrow$
<code>risk_D</code>	dict or None	Bocola two-branch risk inputs for <code>bank_backward</code> (see make_risk_inputs); None $\Rightarrow$ risk-

Returns a dict of all endogenous block outputs: firm/capital/bank/government sub-dicts, dividends, CES price indices, household policies and income paths, aggregate C/A paths, trade flows, and the sequence of start-of-period cross-sectional distributions `D_start_*` (shape (T+1, n_a, n_e)).

Logic — one full economy per call, evaluated block by block in dependency order:

1. **Firms.** [solve_firm_path](#) maps (N, Kap, Z) into Y, w, mpk (purely contemporaneous, both countries).
2. **Capital.** [solve_capital_path](#) inverts the Jermann accumulation technology on the guessed Kap path to obtain investment I, the capital price Q, the realized return rk, and capital-producer profit. Lags `Kap_lag`, `Q_lag` come from `init` (mid-path start) or steady state.
3. **CES price indices.** `trade.ces_price(p)` per period, per country.
4. **Bank backward pass.** `bank.bank_backward` computes, from expected-return FOCs under the priced default probabilities (and optionally the two-branch risk inputs `risk_D`): bond prices `Q_bD`, `Q_bF`, franchise values `a_lpha`, IC multipliers `mu`, discount factors `Omega`, and the *cross-border* bond holdings `b_D_F`, `b_F_D` from portfolio FOCs.
5. **Government.** [govt_transition](#) forward-integrates the debt stock under the Bohn tax rule at the just-computed bond prices, applying any *realized* haircuts. Optional `init` keys `b_gov0_*` (initial stock) and `b_anchor_*` (Bohn-rule anchor) support mid-path and post-default starts.
6. **Bond-market clearing against the true stock.** Domestic banks are the marginal (residual) holders:

$$\begin{aligned} b_{D,D}[t] &= b_{gov_D_eop}[t] - b_{D,F}[t] \\ b_{F,F}[t] &= b_{gov_F_eop}[t] - b_{F,D}[t] \end{aligned}$$

A `RuntimeError` is raised if either residual holding turns non-positive (cross-border FOC holdings exceeding the outstanding stock) — caught by the outer solver and converted into a penalty.

7. **Bank forward pass.** `bank.bank_forward` rolls net worth forward from *realized* returns (marked-to-market bond and capital revaluations, realized haircuts via `def_real_*`), producing `n`, `n_IC` (IC-implied net worth), dividends `div`, and deposit supply `Dep_supply`. Initial bank states and lagged prices come from `init` for mid-path starts.
8. **Dividends to households.** With flexible prices the markup is constant: $Div = (1 - mc) \cdot Y + cap_profit + div_bank$.
9. **Household income.** In composite-good units, per idiosyncratic state `e`:

$$y_t(e) = (w_t / P_{CES_t}) \cdot N_t \cdot e + (Div_t - Tax_t) / P_{CES_t}$$

with the GHH disutility $w_t = \chi \cdot N_t^{(1+1/frisch)} / (1+1/frisch)$ passed separately (it is subtracted inside the EGM composite, not from income).

10. Real deposit returns (Fisher equation, predetermined rate).

$$r_real[t] = (1 + rdep[t-1]) \cdot P_CES[t-1] / P_CES[t] - 1$$

built as a length-T+1 array (the terminal entry uses $P_CES[T] = 1$, i.e. steady state). Period -1 anchors ($rdep_prev$, P_lag) come from `init` when the path starts mid-crisis; the calibration targets otherwise.

11. **Household EGM backward.** `solve_backward_transition` for each country: policies $c[t]$, $a'[t]$ by backward induction from the steady-state terminal condition.
12. **Distribution forward.** Starting from `init["D_*"]` or the stationary distribution: per period, aggregate consumption on the *start-of-period* distribution, then push the distribution forward (`forward_iterate`) and aggregate *end-of-period* assets. All T+1 start-of-period distributions are stored (`D_start_*`) so a default branch can be launched from any base date.
13. **Trade.** `trade.import_demand` per period and `trade.trade_balance` give IM and NX for both countries.

Economic purpose. This is the model's general-equilibrium map: given prices and quantities on the 7 guessed paths, it produces every other endogenous object consistently with agent optimization (banks, households, firms) and government policy. The block ordering embodies the model's causal structure under perfect foresight: prices from marginal conditions (backward passes) precede flows and stocks (forward passes).

Computational aspects.

- No internal iteration: every block is a direct computation given its inputs, so cost is linear in T. The dominant costs are the household EGM ($O(T \cdot n_a \cdot n_e)$) and the distribution forward pass.
- Errors from infeasible guesses (negative Jermann bracket, non-positive bond holdings, NaN powers) are raised as exceptions, not returned as NaN, so the outer solver can penalize immediately.
- The distinction between the bank *backward* pass (expected returns, FOCs, priced default probabilities) and *forward* pass (realized flows, net worth, realized haircuts) is what implements the PRICED vs REALIZED default split.

solve_transition

```
solve_transition(ss, cal, Z_D_path, Z_F_path,
                def_price_D=None, def_price_F=None,
                def_real_D=None, def_real_F=None,
                verbose=True, maxiter=300, y0=None,
                init=None, risk_D=None, try_krylov=True,
                hybr_factor=100.0) -> dict
```

Parameter	Default	Role
<code>y0</code>	flat SS paths	Initial guess for the stacked 7T unknown vector (warm start).
<code>init</code>	None	Mid-crisis initial state (passed through to <code>_inner_economy</code>).
<code>risk_D</code>	None	Bocola risk inputs (risk-neutral if None).
<code>try_krylov</code>	True	Enable the Krylov fallback solver.
<code>hybr_factor</code>	100.0	Initial trust-region size for hybr (small values $\Rightarrow$ cautious steps near penalty walls).
<code>maxiter</code>	300	Scales <code>maxfev</code> for hybr / <code>maxiter</code> for Krylov.

Returns a flat dict of all solved paths: the 7 unknowns, all firm/capital/bank/ government/household/trade outputs (suffixed `_D/_F`), the default paths actually used, and `y_vec` — the solved unknown vector, used as a warm start by every outer loop.

Logic.

1. Build the default initial guess: all seven paths flat at their steady-state values ($N = 1$, $Kap = Kap_{ss}$, $rdep = r_{dep_target}$, $p = p_{ss}$).
2. Solve $residual(y) = 0$ (see below) with `scipy.optimize.root(method="hybr")`, $maxfev = \max(maxiter \cdot (7T+1), 50000)$.
3. **Polish restarts.** If the max-abs residual exceeds $accept_tol = \max(cal["tol_mkt"], 1e-6)$, restart `hybr` up to twice *from the stalled point* with a fresh small trust region ($factor=1.0$). At long horizons `hybr` can stall marginally above tolerance on a stale Jacobian; a restart typically shaves the last order of magnitude. No-op when already converged.
4. **Krylov fallback.** If still above tolerance and `try_krylov`, attempt `root(method="krylov")` from the original y_0 ; keep whichever solution has the smaller residual. Krylov exceptions (Jacobian degeneracy near penalty walls) are caught and ignored.
5. Raise `RuntimeError` if the final residual exceeds $accept_tol$; otherwise re-evaluate `_inner_economy` at the solution and assemble the output dict.

residual (nested in solve_transition)

Maps the stacked unknown vector y (ordering $[N_D \mid N_F \mid Kap_D \mid Kap_F \mid rdep_D \mid rdep_F \mid p]$, each block length T) to $7T$ residuals:

#	Residual (per period, normalized)	Pins
1	Capital market D: $(n_{IC_D} - n_D) / n_{ss_D}$	Kap_D
2	Capital market F: $(n_{IC_F} - n_F) / n_{ss_F}$	Kap_F
3	Labour market D: $(\chi_D \cdot N_D^{(1/frisch)} - w_D/P_{CES_D}) / (w_D/P_{CES_D})$	N_D
4	Labour market F: same, country F	N_F
5	Deposit market D: $(P_{CES_D} \cdot A_D - Dep_supply_D) / Kap_D_{ss}$	$rdep_D$
6	Deposit market F: same, country F	$rdep_F$
7	Goods market D: $(Y_D - P_{CES_D} \cdot C_D - I_D - NX_D - G_D) / Y_{ss_D}$	p

Notes on the residuals:

- The capital-market condition equates net worth implied by the binding incentive constraint (n_{IC} , from the backward pass) with net worth accumulated from realized returns (n , from the forward pass) — the GK equilibrium condition that ties asset demand to bank equity.
- The labour market is the GHH static FOC ($\chi \cdot N^{(1/frisch)} = w/P_{CES}$), which under GHH is independent of consumption.
- Deposits: the bank supplies Dep_supply in nominal home-good units while household assets A are in real composite units — hence the P_{CES} conversion.
- Goods market F and the current account are *not* imposed (Walras).

Robustness devices inside residual:

- **Domain guard:** $p \leq 0.05, N \leq 0.01, Kap \leq 0.1$ return a flat penalty vector `np.full(7T, 10.0)` before any computation (fractional powers would go NaN below these thresholds).
- **Uniform wall height:** every failure path (guard, exception in `_inner_economy`, non-finite residuals) returns the *same* penalty height 10.0. Unequal walls would bias `hybr`'s finite-difference gradient toward the lower wall, steering it into NaN territory.

Economic purpose. This is the model's equilibrium concept: a perfect-foresight path on which banks' incentive constraints, the GHH labour FOC, deposit-market clearing, and the D goods market all hold every period, with government debt endogenous throughout. Solving all periods simultaneously (rather than shooting) handles the strong forward-looking linkages: bond prices at t depend on the whole future default-probability and return path through the bank backward pass.

Computational aspects.

- Problem size 7T (3,500 unknowns at T=500). `hybr` builds a dense finite-difference Jacobian, so each Newton iteration costs $O(7T)$ inner-economy evaluations; the whole TFP experiment runs in about a minute.
- `y_vec` in the output enables warm starting: all outer loops (`solve_transition_ck`, `solve_transition_ck_risk`, `homotopies`) restart the Newton from the previous solution, which usually converges in a handful of iterations.
- Zero-shock regression: with flat `Z` paths the solver must stay at the steady state to $\leq 1e-5$ (enforced in `tests/test_transition_walras.py`).

`solve_transition_ck`

```
solve_transition_ck(ss, cal, Z_D_path, Z_F_path,
                  sunspot_D_path=None, sunspot_F_path=None,
                  def_real_D=None, def_real_F=None,
                  verbose=True, y0=None) -> dict
```

Returns the `solve_transition` output dict plus `sunspot_D`, `sunspot_F`.

Logic. The Cole-Kehoe *risk-neutral* wrapper: a fixed-point iteration on the crisis-zone indicator.

1. Initialize the priced default probability at steady-state debt: `def_price[t] = ck_default_prob(B_gov_ss, Y_ss, cal, sunspot[t])`.
2. Repeat (up to `cal["ck_max_iter"]`, damping `cal["ck_damping"]`):
 1. Solve the full transition with the current `def_price` paths (warm-started from the previous round's `y_vec`).
 2. Re-evaluate the crisis zone on the *solved* beginning-of-period debt path: `def_price_new[t] = ck_default_prob(b_gov_bop[t], Y_ss, cal, sunspot[t])`.
 3. Stop when `max|def_price_new - def_price| < cal["ck_tol"]`; otherwise damp and iterate.

Economic purpose. Implements Bocola's pass-through experiment in Cole-Kehoe form: the sunspot $\xi_t \in [0, 1]$ is the probability that lenders coordinate on the no-rollover equilibrium *conditional on the crisis zone being active*. It is priced into bonds but (in the risk-only baseline) never realized: bank net worth falls purely through mark-to-market bond repricing. The only fixed point beyond the transition itself is the zone indicator, because `def_price` depends on whether the endogenous debt-to-output ratio crosses `b_ck_low` / `b_ck_high`.

Computational aspects. When debt stays interior to the crisis zone (the calibrated configuration: steady-state debt/quarterly-output ≈ 3.7 lies between `b_ck_low_D = 3.0` and `b_ck_high_D = 6.0`), the zone indicator never changes and the map converges in one iteration — the loop is effectively a single warm-started transition solve plus a consistency check.

4. Module `risk_branch`

File: `code/global/risk_branch.py`

Implements the **Bocola (2016) risk channel** on top of the risk-neutral CK solver. Bankers discount with the household SDF and hold state-contingent continuation values: news that default is more likely pairs *high* marginal valuations (the default state is a recession with depressed bank net worth) with *low* asset returns — a covariance premium on both sovereign bonds and capital claims, i.e. precautionary deleveraging even when funding is cheap.

The perfect-foresight implementation ("R2-lite") averages two branches at each base date `t`: with probability $1 - \pi_{t+1}$ the economy continues on the base path; with probability π_{t+1} a haircut is realized at `t+1` and the economy jumps to a post-default transition. **One representative branch** (default at $\tau^* = 1$, launched from the impact state) is solved and its period-0 objects are reused at every base date. Documented approximations: $\Lambda^{nd} \equiv \text{beta_inter}$ on the base path (the channel lives in $\pi \cdot (\Omega^{nd} - \Omega^{nd})$), branch state-dependence across base dates treated as second order, and household π -blindness (the deposit Euler never weights the default branch — risk pricing lives entirely in the bank block). Setting $\pi \equiv 0$ nests the risk-neutral model exactly (regression-tested in `tests/test_risk_channel.py`).

extract_init_state

`extract_init_state(out, ss, cal, tau) -> dict`

Logic. Builds the `init` dict needed to launch a transition at base period `tau`, using only period `tau - 1` objects of the solved base path out:

- household cross-sectional distributions `D_D, D_F (= D_start[tau])`;
- bank states per country: lagged net worth `n_prev`, portfolio shares `kappa_prev = Q·Kap/n_IC`, `phi_bdom_prev = Q_b·b_dom/n_IC`, `phi_bfor_prev` (cross-border leg, converted at `p`), and the predetermined deposit rate `rdep_prev`;
- initial government stocks `b_gov0_*` (end-of-period at `tau - 1`);
- all price/stock lags the inner economy needs: `Kap_lag, Q_lag, Q_bD_lag, Q_bF_lag, p_lag, P_lag_*`.

Economic purpose. The state vector of the economy at a point mid-path — the mechanism by which default branches (and, in future work, policy interventions) can start from a crisis state rather than the steady state.

Computational aspects. Pure indexing; the only subtlety is that portfolio shares are normalized by `n_IC` (the IC-implied net worth), matching the convention `bank_forward` uses to reconstruct balance sheets. Two small helpers support branch launches: `_shift_path(x, tau, T)` (shift a path forward by `tau`, padding with the final value) and `_shifted_y0(out, tau, T)` (the shifted base solution as a Newton warm start).

solve_default_branch

`solve_default_branch(out, ss, cal, tau=1, verbose=False, y0=None, target_scale=None) -> dict`

Returns a full `solve_transition` output dict for the post-default economy, plus `haircut_scale` ($\equiv 1.0$; the feasibility ladder was removed 2026-07-13 — at calibrated recovery rates the haircut costs banks only a modest share of their sovereign exposure and never threatens equity, so the largest-feasible-event search was moot; `target_scale` is retained for caller compatibility and unused).

Logic.

1. Extract the base-path state entering period `tau` via [extract_init_state](#).
2. Impose the full haircut at branch period 0: `def_real_D[0] = 1`, so bonds pay `recovery_rate_D` of face value once.
3. Apply the canonical **output cost of default** (Arellano 2008 tradition):

$$Z_D_branch[h] = Z_D_base[tau + h] \cdot (1 - cost \cdot \rho_c^h)$$

with `cost = def_output_cost_D`, `$\rho\_c = def\_output\_rho\_D$` . This is what makes the default state a recession — the source of the high marginal valuations Λ^d , α^d that give the risk premium its sign.

4. **Re-anchor the Bohn rule** to the post-haircut stock: `b_anchor_D = b_gov0_D · recovery_rate_D`. Keeping the steady-state anchor would turn the haircut into a large tax-cut windfall ($\varnothing \cdot (b - b_{ss}) \ll 0$), making default *expansionary* and flipping the risk premium's sign.
5. Solve the branch transition with `solve_transition` (warm start: previous round's branch solution if available, else the shifted base path; `try_krylov=False`). If `hybr` stalls on the alpha-explosion penalty wall (poisoned finite-difference Jacobian), retry once with a small trust region (`hybr_factor = 0.1`). Raise `RuntimeError` if both fail (bank equity wiped out — recapitalization is not modelled).
6. **Absorbing-branch check:** warn if post-default debt `b_post / Y_ss ≥ b_ck_low_D` — the branch is assumed to carry no further default risk, which requires debt to exit the crisis zone.

Economic purpose. The representative “feared” state: what the economy looks like if lenders’ no-rollover coordination materializes. Its period-0 asset returns and valuations are the *default-branch payoffs* that the base-path bankers price; the wedge between them and base-path payoffs is the entire risk channel.

Computational aspects. Each branch solve is a full 7T Newton problem, so it dominates the cost of a risk round. Warm-starting from the previous round’s branch solution makes rounds after the first much cheaper.

make_risk_inputs

```
make_risk_inputs(branch, base, ss, cal) -> dict
```

Returns the `risk_D` dict consumed by `bank_backward` (without the probability `pi`, which the outer loop attaches): default-branch discount loadings `Omega_d_D`, `Omega_d_F` (shape `(T,)`), branch period-0 prices/returns `rk_d_D`, `rk_d_F`, `Q_bD_d`, `Q_bF_d`, `p_d` (scalars), and the survival factor of the priced event `surv_d = 1 - haircut_scale · (1 - recovery_rate_D)`.

Logic. The banker’s default-state discount factor is

$$\Omega^d_X[t] = \Omega^d_X[t] \cdot [(1 - f_X) + f_X \cdot \alpha^d_X(0)], \quad X \in \{D, F\}$$

where $\alpha^d(0)$ is the branch’s period-0 franchise value and f the banker survival probability. The default-state SDF Ω^d has three variants (`cal["sdf_mode"]`):

Mode	Ω^d	Status
"income" (baseline)	$\text{beta_inter} \cdot (Y^d(0) / Y^{nd}_{t+1})^{(-\sigma)}$ — Euler-consistent loading on aggregate <i>output</i> .	Robust rep-agent
"empirical"	$\text{beta_inter} \cdot \text{kappa_d}$ (free loading <code>cal["kappa_d"]</code>).	Bocola’s asset-pr
"model"	From GHH consumption composites $x = C - v(N)$, branch vs base.	Wrong-signed u

The F-banker loads on F output in the same D-default event; the contagion recession there is small, so that leg stays near 1.

Economic purpose. Translates the solved default branch into the two numbers per asset the bank pricing FOCs need: how much the banker *values* payoffs in the default state (Ω^d) and what those payoffs *are* (rk^d , Q^d , $surv_d$). Because $\Omega^d > \Omega^{nd}$ while default-state payoffs are low, expected-discounted returns fall and bond/capital prices are depressed beyond actuarially fair default compensation — the risk premium.

Computational aspects. Cheap (array arithmetic). The sign gate exists because the income-SDF must never invert (a positive-output branch would make sovereign risk expansionary); it prints a warning and switches modes for the round rather than failing.

solve_transition_ck_risk

```
solve_transition_ck_risk(ss, cal, Z_D_path, Z_F_path,
                        sunspot_D_path=None, sunspot_F_path=None,
                        verbose=True, max_rounds=6, damp=0.5, tol=1e-3,
                        y0=None) -> dict
```

Returns the base-path `solve_transition` dict plus branch (the full default-branch dict), `risk_D_inputs`, updated `def_price_*`, `sunspot_*`, and `risk_converged` (bool).

Logic. The outer fixed point between the base path and the representative branch:

Round 0: risk-off base path (`solve_transition_ck` — also settles the crisis-zone indicator; `y0` may come from a pre-solved run)
Round $k = 1 \dots \text{max_rounds}$:
1. Solve the representative default branch from the base impact state

```

(solve_default_branch, tau=1; warm-started).
2. Convert branch objects to risk inputs (make_risk_inputs) and DAMP:
   risk_in ← (1-damp)·risk_in + damp·new_in          (damp = 0.5)
3. Convergence metric on the branch objects:
   conv = max( |ΔQ_bD_d|/Q_bD_ss, |Δrk_d_D|, mean|ΔQ^d_D|/beta_inter )
4. Attach the priced probability π = min(chi_tilt·def_price_D, 1) and
   re-solve the base path with two-branch pricing (solve_transition with
   risk_D; warm-started from the previous y_vec).
5. Refresh the crisis-zone indicators on the new debt paths.
6. Stop when conv < tol AND k ≥ 2.
Final:   zone-consistency loop (≤ 5 re-solves) if the refreshed indicator
        differs from the one used in the last base solve – a no-op when
        debt stays interior to the crisis zone.

```

Failure handling: if a branch re-solve fails after round 1, the previous round's risk inputs are kept and the loop exits with `risk_converged = False`; if the zone indicator never settles in the final consistency step, a warning is printed and the flag is cleared.

Economic purpose. The centerpiece experiment solver. The fixed point is economically necessary: the branch is launched from the *base* impact state (which depends on pricing), while base pricing depends on branch objects. At convergence, bankers' two-branch expectations are consistent with the post-default economy they fear, and the crisis-zone indicator is consistent with the equilibrium debt path.

Computational aspects. Cost per round $\approx$ one branch solve + one warm-started base solve (the diagnostics print per-round timings). Damping at 0.5 stabilizes the branch $\leftrightarrow$ base map; the $rd \geq 2$ requirement prevents spurious one-round "convergence" before damping has acted. `chi_tilt` (calibration, default 1.0 = physical probabilities) is an EZ-lite pessimism dial on the priced probability.

bond_decomposition

```
bond_decomposition(out, ss, cal) -> dict
```

Returns (τ ,) arrays in annualized basis points (deviations from steady state where applicable): `total_yield`, `defcomp`, `risk`, `liquidity`, `promised_excess`.

Logic. An *exact* per-period identity splitting the D-bond promised excess return:

$$\begin{aligned}
 \text{payoff}^{\text{nd}}/Q - 1 - r_{\text{dep}} &= (\text{payoff}^{\text{nd}} - E[\text{payoff}]) / Q && [\text{default compensation}] \\
 &+ (E[\text{payoff}]/Q - 1 - r_{\text{dep}} - \lambda\mu/\tilde{\Omega}) && [\text{risk premium}] \\
 &+ \lambda\mu/\tilde{\Omega} && [\text{liquidity premium}]
 \end{aligned}$$

where $\text{payoff}^{\text{nd}} = \delta_b + (1-\delta_b) \cdot Q'$ is the no-default HM payoff, $\text{payoff}^{\text{d}} = \text{surv}_d \cdot (\delta_b + (1-\delta_b) \cdot Q^{\text{d}})$ the default-branch payoff, and $E[\text{payoff}] = (1-\pi') \cdot \text{payoff}^{\text{nd}} + \pi' \cdot \text{payoff}^{\text{d}}$ the physical expectation. The liquidity term $\lambda\mu/\tilde{\Omega}$ is the incentive-constraint (balance-sheet) spread from the bank block. In risk-neutral mode the middle term is 0 *by the pricing equation*; with the Bocola channel it is positive because $\Omega^{\text{d}} > \Omega^{\text{nd}}$ depresses Q . The headline `total_yield` is the perpetuity yield deviation $\delta_b/Q - \delta_b$ vs steady state; the branch price Q^{d} and `surv_d` are taken from `out["risk_D_inputs"]` when present, otherwise the recovery-rate shortcut applies.

Economic purpose. The paper's key diagnostic: how much of the sovereign spread is actuarial default compensation vs a true risk premium vs bank balance-sheet tightness — the model analogue of Bocola's decomposition (risk channel share of the lending-spread response, benchmark "up to 45%").

Computational aspects. Pure post-processing; the identity is regression-tested (`tests/test_risk_channel.py`) so any change to bank pricing that breaks the decomposition fails loudly. Annualization: quarterly rate $\times 4e4 \rightarrow$ bps/yr.

5. Module household

File: code/global/household.py

One-asset incomplete-markets consumption-savings block with GHH preferences, solved by the endogenous grid method (EGM, Carroll 2006). Utility is $u(x) = x^{1-\sigma}/(1-\sigma)$ over the composite $x = c - v(N)$, $v(N) = \chi \cdot N^{(1+1/\text{frisch})}/(1+1/\text{frisch})$. GHH kills the wealth effect on labour supply: the labour FOC is static and handled in the transition solver, so the household block only chooses consumption/savings.

make_asset_grid

`make_asset_grid(cal, country="D") -> (n_a,) array`

Power-spaced grid $a_{\min} + (a_{\max} - a_{\min}) \cdot \text{linspace}(0, 1, n_a)^{\text{curve}}$ — with $\text{curve} > 1$, points concentrate near the borrowing constraint where policy functions have the most curvature. Current calibration: 250 points on $[0, 87.2]$, curvature 2.

egm_step

`egm_step(c_next, a_grid, Pi, r_today, r_next, y_e, beta, sigma, a_min, vN_today=0.0, vN_next=0.0) -> (c_today, a_pol_today)`

Input	Shape	Role
c_next	(n_a, n_e)	Next-period consumption policy on the asset grid.
Pi	(n_e, n_e)	Markov transition matrix for idiosyncratic productivity.
r_today, r_next	scalar	Real deposit returns received at t and t+1 (predetermined-rate, Fisher-adjusted).
y_e	(n_e,)	Non-asset income by productivity state (composite units).
vN_today, vN_next	scalar	GHH labour disutility at t, t+1.

Logic — one backward EGM step:

1. GHH composite tomorrow: $x' = \max(c' - vN', 1e-11)$.
2. Expected marginal utility: $E[u'(x')] = (x'^{-\sigma}) \Pi'$ (row-wise over e).
3. Invert the Euler equation on the *savings* grid:

$$x'^{-\sigma} = \beta \cdot (1 + r') \cdot E[x'^{-\sigma}] \quad \Rightarrow \quad x_{\text{endo}} = (\beta(1+r') \cdot E[u'])^{(-1/\sigma)}$$

4. Recover consumption $c_{\text{endo}} = x_{\text{endo}} + vN_{\text{today}}$ and the endogenous asset grid from the budget constraint:

$$a_{\text{endo}} = (c_{\text{endo}} + a' - y(e)) / (1 + r_{\text{today}})$$

5. Per productivity state, linearly interpolate ($a_{\text{endo}} \rightarrow c_{\text{endo}}$) back onto the fixed grid; savings follow from the budget: $a' = (1+r)a + y - c$.
6. **Borrowing constraint:** grid points below $a_{\text{endo}}[0, e]$ are constrained — set $a' = a_{\min}$ and let consumption absorb the rest; finally clip $a' \geq a_{\min}$.

Economic purpose. The household Euler equation under incomplete markets: the consumption function embeds precautionary savings against idiosyncratic income risk (via the expectation over Π), which generates the wealth distribution and the aggregate deposit supply the banks intermediate.

Computational aspects. EGM avoids any root-finding: the Euler equation is inverted analytically and the only numerical operation is a 1-D interpolation per income state. The $1e-11$ floor on x' guards the fractional power near the constraint. Cost $O(n_a \cdot n_e)$ per step; fully vectorized except the per-state interpolation loop ($n_e = 2$ currently, so negligible).

solve_steady_state_household

```
solve_steady_state_household(a_grid, Pi, r_ss, y_e, beta, sigma, a_min, tol,
                             maxiter=10_000, vN_ss=0.0) -> (c, a_pol)
```

Logic. Time-iterates [egm_step](#) with constant (r, y, vN) from the initial guess $c = (1+r)a + y - a$ (consume the annuity) until $\max|c_{\text{new}} - c| < \text{tol}$ (`cal["tol_hh"] = 1e-12`); raises `RuntimeError` on non-convergence.

Economic purpose. The stationary consumption/savings policy used (i) inside the steady-state solver to clear the deposit market (the β calibration stage) and (ii) as the *terminal condition* of every transition backward pass.

solve_backward_transition

```
solve_backward_transition(a_grid, Pi, r_path, y_path, c_ss, beta, sigma, a_min,
                         vN_path=None) -> (c_path, a_pol_path) # (T, n_a, n_e)
```

Input	Shape	Note
<code>r_path</code>	$(T+1,)$	Real returns; entry $t+1$ is the return relevant for the period- t Euler equation.
<code>y_path</code>	(T, n_e)	Income by period and productivity state.
<code>c_ss</code>	(n_a, n_e)	Terminal condition: steady-state consumption policy.
<code>vN_path</code>	$(T,)$	GHH disutility path (zeros if <code>None</code>).

Logic. Backward induction $t = T-1, \dots, 0$, each step one [egm_step](#) with $(r_{\text{today}}, r_{\text{next}}) = (r_{\text{path}}[t], r_{\text{path}}[t+1])$. The terminal GHH disutility is $vN_{\text{next}} = vN_{\text{path}}[-1]$ — period T is permanently at steady state, and using 0 instead would mis-state the composite and make households under-save.

Economic purpose. Household expectations under perfect foresight: policies at every t are consistent with the entire future path of returns and incomes. Note the documented **π -blindness**: the deposit Euler never weights the default branch, so there is no precautionary-savings response to default-state income risk — risk pricing lives entirely in the bank block (see `risk_branch` docstring).

Computational aspects. $O(T \cdot n_a \cdot n_e)$; with $T=500$, 250 grid points and 2 income states this is a small share of an inner-economy evaluation. Policies are stored densely for the subsequent distribution forward pass.

6. Module firms

File: `code/global/firms.py`

Cobb-Douglas production with monopolistic competition and *fully flexible* prices — a deliberate benchmark choice (see `CLAUDE.md` "Known limitations"): the markup is constant, so the block is purely static and contemporaneous.

markup_ss

```
markup_ss(cal, country="D") -> float
```

Real marginal cost under flexible prices: $mc = (\varepsilon - 1)/\varepsilon$ with $\varepsilon = \text{cal}["\text{epsilon}_*"]$ the demand elasticity ($\varepsilon = 6 \Rightarrow mc = 5/6$, a 20% markup). The complement $1 - mc$ is the profit share rebated to households as dividends.

steady_state_firm

steady_state_firm(cal, Kap_ss, country="D") -> dict

Logic. With $N_{ss} = 1$ normalized:

$Y_{ss} = Z_{ss} \cdot K_{ss}^\alpha$	(Cobb-Douglas, $N = 1$)
$w_{ss} = mc \cdot (1-\alpha) \cdot Y_{ss}$	(labour demand FOC)
$mpk_{ss} = mc \cdot \alpha \cdot Y_{ss} / K_{ss}$	(capital demand FOC)
$I_{ss} = \delta \cdot K_{ss}, \quad C_{ss} = Y_{ss} - I_{ss} - G$	
$chi = w_{ss} / N_{ss}^{(1/frisch)}$	(GHH static FOC $\Rightarrow$ pins $N_{ss} = 1$)

Economic purpose. Steady-state firm block used by steady_state.py; the returned chi overwrites the calibration warm start so that labour-market clearing at $N_{ss} = 1$ holds by construction (similarly Z_{ss} is rescaled upstream to normalize $Y_{ss} = 1$).

solve_firm_path

solve_firm_path(N_path, Kap_path, Z_path, cal, country="D") -> dict
returns Y, w, mpk (each (T,)) and mc (scalar)

Logic. Vectorized contemporaneous evaluation along the path:

$Y_t = Z_t \cdot K_t^\alpha \cdot N_t^{(1-\alpha)}$
$w_t = mc \cdot (1-\alpha) \cdot Y_t / N_t$
$mpk_t = mc \cdot \alpha \cdot Y_t / K_t$

Economic purpose. Supplies output (goods-market resource constraint), the wage (labour-market residual and household income), and the marginal product of capital (input to the Jermann return calculation in capital). Note the timing convention: production at t uses the *contemporaneous* guessed Kap_t — the capital block handles the lag structure of returns.

Computational aspects. Pure NumPy arithmetic, no state; numerically safe as long as the transition solver's domain guard keeps N and Kap positive.

7. Module government

File: code/global/government.py

Three ingredients: **Hatchondo-Martinez (2009)** geometric-decay perpetuity bonds, the **Bohn (1998)** tax rule, and **Cole-Kehoe (2000)** self-fulfilling crisis zones.

hm_bond_price_ss / hm_bond_return_ss

hm_bond_price_ss(rdep_ss, delta_b)	-> $Q_{B_{ss}} = \text{delta_b} / (\text{rdep_ss} + \text{delta_b})$
hm_bond_return_ss(Q_B_ss, delta_b)	-> $(\text{delta_b} + (1-\text{delta_b}) \cdot Q_{B_{ss}}) / Q_{B_{ss}} - 1$

The HM perpetuity pays coupon δ_b per unit of face value and the stock decays at rate $1 - \delta_b$; Macaulay duration $\approx 1/\delta_b$ quarters. The steady-state price discounts the coupon stream at $rdep_{ss}$; the realized return equals $rdep_{ss}$ by no-arbitrage (a consistency identity checked in tests/test_ss_identities.py). The long duration is what converts a *priced* default probability into large mark-to-market losses on bank balance sheets.

govt_steady_state

```
govt_steady_state(cal, rdep_ss, country) -> dict
# Q_B_ss, rb_ss, Tax_ss, b_gov_ss, coupon_ss
```

Logic. With a constant stock B_{gov_ss} and no default risk, maturing principal is rolled over: issuance of $\delta_b \cdot B_{gov}$ new bonds at price Q_B_{ss} each period, giving the balanced-budget tax

$$Tax_{ss} = G + coupon_{ss} \cdot (1 - Q_B_{ss}), \quad coupon_{ss} = \delta_b \cdot B_{gov_ss}$$

(the coupon bill net of rollover proceeds). Used by `steady_state.py` and as the anchor for the transition Bohn rule.

ck_default_prob

```
ck_default_prob(b_gov, Y_ss, cal, sunspot, country) -> float
```

Logic. Piecewise in the debt-to-quarterly-output ratio b/Y_{ss} :

```
b/Y < b_ck_low          → 0          (safe zone)
b_ck_low ≤ b/Y < b_ck_high → sunspot (crisis zone: self-fulfilling risk)
b/Y ≥ b_ck_high         → 1          (certain default)
```

Economic purpose. The Cole-Kehoe crisis-zone map. `sunspot` is the exogenous probability that lenders coordinate on the no-rollover equilibrium *conditional on the crisis zone* — the analogue of Bocola's exogenous AR(1) default-risk process (his eq. 12) restricted to the CK zone. In the risk-only experiment this probability is priced but never realized. Calibration places the steady state inside the D crisis zone (so the sunspot is priced immediately) with `b_ck_high` out of reach (no fundamental default), and country F always safe (`b_ck_low_F` = 99).

Computational aspects. The discontinuities at the thresholds are why the outer solvers iterate on the *zone indicator* rather than differentiating through it: `def_price` is held fixed inside each Newton solve and refreshed between solves.

govt_transition

```
govt_transition(cal, gs, Q_B_path, def_real_path, country,
               b_gov0=None, b_anchor=None) -> dict
# Tax, coupon, net_issuance, b_gov (bop), b_gov_eop - all (T,)
```

Parameter	Default	Role
<code>gs</code>	—	Country's <code>govt_steady_state</code> dict (anchors).
<code>Q_B_path</code>	—	Bond price path from <code>bank_backward</code> (taken as given).
<code>def_real_path</code>	<code>zeros</code>	Realized haircut indicator per period.
<code>b_gov0</code>	<code>b_gov_ss</code>	Initial stock (mid-path starts).
<code>b_anchor</code>	<code>b_gov_ss</code>	Bohn-rule anchor; must be re-set to the post-haircut stock on default branches.

Logic. One forward pass over the budget identity; for each t :

```
Tax_t      = Tax_base + φ · (b_t - b_anchor)          (Bohn rule)
surv_t     = 1 - def_real_t · (1 - recovery_rate)
coupon_t   = δ_b · b_t · surv_t
new_bonds  = (G + coupon_t - Tax_t) / Q_t            (deficit financed at market price)
b_{t+1}    = (1 - δ_b) · b_t · surv_t + new_bonds
```

with $\text{Tax_base} = \text{Tax_ss}$ at the steady-state anchor, or the budget-balancing tax at a custom anchor ($G + \delta_b \cdot b_{\text{anchor}} \cdot (1 - Q_{B_ss})$). Realized haircuts scale both the coupon and the surviving principal by surv_t .

Economic purpose. Fiscal policy and debt dynamics. Two properties are load-bearing:

- **Endogenous issuance at market prices** is the CK/Bocola amplification leg: when beliefs depress Q , financing a given deficit requires *more* face value, which constrained banks must absorb — raising debt and (potentially) the priced probability further.
- **Anchor re-basing on default branches** prevents the haircut from becoming a tax-cut windfall through $\phi \cdot (b - b_{ss}) < 0$, which would make default expansionary and flip the sign of the risk premium (see [solve_default_branch](#)).

Computational aspects. A single $O(T)$ scalar recursion — deliberately with *no feedback from taxes to prices inside the block*; all price feedback runs through the outer Newton (bond prices are recomputed by `bank_backward` at every residual evaluation, and this block is re-integrated against them). This is what removes the need for a separate outer debt loop.

8. Module capital

File: `code/global/capital.py`

Capital producers with the **Jermann (1998)** concave accumulation technology

$$K_t = (1 - \delta) \cdot K_{t-1} + \Gamma(\iota_t) \cdot K_{t-1}$$

$$\Gamma(\iota) = \gamma_0 \cdot \iota^{(1-\xi)} + \gamma_1, \quad \iota = I / K_{t-1}$$

which makes the price of installed capital Q move with investment — the asset-price channel through which bank deleveraging raises firms' cost of capital.

gamma_params

```
gamma_params(cal, country="D") -> (gamma0, gamma1)
# gamma0 =  $\delta^\xi / (1-\xi)$ , gamma1 =  $-\delta \cdot \xi / (1-\xi)$ 
```

Pins the adjustment-cost function so that at the steady state ($\iota = \delta$): $\Gamma(\delta) = \delta$ (no adjustment cost) and $\Gamma'(\delta) = 1$ (hence $Q_{ss} = 1$). The curvature $\xi = \text{cal}["\text{ksi}^*"]$ controls the elasticity of Q to investment.

capital_demand

```
capital_demand(rk_ss, mc_ss, cal, country="D") -> K_ss
# K_ss =  $(mc \cdot \alpha \cdot Z_{ss} / (rk_{ss} + \delta))^{1/(1-\alpha)}$ 
```

Inverts the steady-state capital FOC $\text{mpk}_{ss} = rk_{ss} + \delta$ (with $Q_{ss} = 1$ and $N_{ss} = 1$) to get the capital stock consistent with a guessed required return — used by the steady-state solver's capital-market stage.

solve_capital_path

```
solve_capital_path(Kap_path, Kap_ss, Q_ss, mpk_path, cal, country="D") -> dict
# iota, Q, rk, I, cap_profit - all (T,)
```

(The second and third arguments are the *lagged* capital stock and Q entering period 0 — steady-state values on a standard run, init values on a mid-path branch.)

Logic.

1. Invert the accumulation technology on the guessed capital path:

$$\begin{aligned} \text{bracket}_t &= (K_t/K_{t-1} - (1-\delta) - \gamma_1) / \gamma_0 \\ \iota_t &= \text{bracket}_t^{1/(1-\xi)} \end{aligned}$$

A negative bracket (capital falling faster than the adjustment technology allows) raises `ValueError` **immediately** — fractional powers of negatives return NaN silently, so the explicit raise lets the transition solver penalize the guess without running the full inner economy on NaNs.

2. Price of capital from the capital producer's FOC (marginal Tobin's Q):

$$Q_t = 1 / \Gamma'(\iota_t) = 1 / (\gamma_0 \cdot (1-\xi) \cdot \iota_t^{-(\xi)})$$

3. Realized return on capital claims held from $t-1$ to t :

$$rk_t = (\text{mpk}_t + (1-\delta) \cdot Q_t) / Q_{t-1} - 1$$

4. Investment $I_t = \iota_t \cdot K_{t-1}$ and the capital-producer profit rebated to households:

$$\text{cap_profit}_t = Q_t \cdot (K_t - (1-\delta)K_{t-1}) - I_t + \text{mpk}_t \cdot (K_t - K_{t-1})$$

Economic purpose. rk is the return on the claims banks hold against firms — the object whose spread over the deposit rate is the model's *lending spread*. Because Q is forward-determined by the whole capital path, mark-to-market revaluations of capital claims hit bank net worth alongside sovereign-bond revaluations, and the Jermann curvature converts bank deleveraging into investment declines rather than pure price adjustment.

Computational aspects. Fully vectorized $0(T)$; the only failure mode is the negative-bracket domain error, which is part of the transition solver's uniform penalty-wall scheme. Note the *inversion* structure: the solver guesses K and this block backs out I and Q , rather than integrating K forward from I .

9. Module distribution

File: code/global/distribution.py

Non-stochastic simulation of the household cross-sectional distribution over (a, e) using the **Young (2010) lottery method**: off-grid savings choices are split across the two neighbouring grid points with weights that preserve the mean exactly.

get_lottery_weights

```
get_lottery_weights(a_pol, a_grid) -> (idx_lo, idx_hi, w_lo, w_hi)
```

Logic. Clip the policy to the grid range; find bracketing indices with `searchsorted` (clipped to `[1, n_a-1]` so every point has a valid pair); set

```
w_hi = (a' - a_grid[lo]) / (a_grid[hi] - a_grid[lo]),    w_lo = 1 - w_hi
```

(guarding zero-width brackets). By construction $w_{lo} \cdot a_{grid}[lo] + w_{hi} \cdot a_{grid}[hi] = a'$, so aggregate assets are preserved without approximation error from the histogram representation.

forward_iterate

```
forward_iterate(D, a_pol, a_grid, Pi) -> D_next    # (n_a, n_e)
```

Logic. One period of the distributional law of motion:

1. **Asset transition (scatter):** for each income state e , deposit each mass point's $D[i, e]$ onto idx_{lo}/idx_{hi} with the lottery weights, using `np.add.at` (unbuffered scatter-add, required because indices repeat).
2. **Income transition (mix):** post-multiply by the Markov matrix: $D_{next} = pre @ Pi$.

Economic purpose. The Kolmogorov-forward step: given today's distribution and policies, tomorrow's distribution. Composed over t it turns individual policies into the aggregate consumption and deposit-supply paths that enter market clearing — the "HA" in HANK.

Computational aspects. $O(n_a \cdot n_e)$ per period plus an $(n_a \times n_e) \cdot (n_e \times n_e)$ matmul. Mass is conserved exactly (weights sum to 1, Pi is a stochastic matrix). The per- e Python loop is negligible at $n_e = 2$.

stationary_distribution

```
stationary_distribution(a_pol, a_grid, Pi, pi_e_stationary, tol,
                      maxiter=100_000) -> D    # (n_a, n_e)
```

Logic. Fixed-point iteration of `forward_iterate` under the *steady-state* policy, initialized as a point mass at `a_min` distributed across income states by the ergodic distribution `pi_e_stationary` (Rouwenhorst). Converges when $\max |D_{new} - D| < tol$ (call `"tol_dist"` = $1e-12$); raises `RuntimeError` otherwise.

Economic purpose. The invariant wealth distribution — the initial condition `D_start[0]` of every transition and the weighting measure for steady-state aggregates (deposit-market clearing pins β per country in `steady_state.py`).

aggregate_assets / aggregate_consumption

```
aggregate_assets(D, a_grid) =  $\sum_{\{i,e\}} D[i,e] \cdot a_{grid}[i]$ 
aggregate_consumption(D, c_pol) =  $\sum_{\{i,e\}} D[i,e] \cdot c_{pol}[i,e]$ 
```

Distribution-weighted sums. Timing convention in `_inner_economy`: consumption is aggregated against the *start-of-period* distribution (the population that consumes at t), assets against the *end-of-period* distribution (deposits carried into $t+1$, matching the bank's funding leg).

10. Call-graph summary

```
main.py
├─ get_calibration()                calibration
├─ solve_steady_state(cal)          steady_state
│   └─ [uses: firms.steady_state_firm, capital.capital_demand,
│       government.govt_steady_state, household.make_asset_grid /
│       solve_steady_state_household,
│       distribution.stationary_distribution, bank.*]
├─ run_tfp: solve_transition(...)    transition
├─ run_sunspot: solve_transition_ck_risk(...) risk_branch
│   └─
│       └─ solve_transition_ck(...)  transition (round 0, risk-off)
│           └─ solve_transition(...) [iterated: zone fixed point]
├─ solve_default_branch(...)        risk_branch [iterated: rounds]
│   └─
│       └─ extract_init_state(...)
│           └─ solve_transition(..., init, def_real_D)
├─ make_risk_inputs(...)            risk_branch
├─ solve_transition(..., risk_D)    transition (two-branch re-solve)
├─ bond_decomposition(...)          risk_branch (spread-decomposition
│                                   figure: plots.plot_bond_decomposition)
```

`solve_transition` (every residual evaluation):

```
_inner_economy
├─ firms.solve_firm_path            (×2 countries)
├─ capital.solve_capital_path       (×2)
├─ trade.ces_price
├─ bank.bank_backward               (prices, FOC holdings; def_price, risk_D)
├─ government.govt_transition        (×2; debt forward-integrated, Bohn tax)
├─ [bond clearing: b_dom = b_gov_eop - b_foreign]
├─ bank.bank_forward                (net worth, dividends, deposits; def_real)
├─ household.solve_backward_transition (×2; EGM)
├─ distribution.forward_iterate / aggregates (×2, over t)
├─ trade.import_demand / trade_balance
```

Regression anchors (run before and after touching any of these functions): `tests/test_ss_identities.py`, `tests/test_bank_block.py`, `tests/test_transition_walras.py`, `tests/test_signs_bocola.py`, `tests/test_risk_channel.py`. Acceptance thresholds are listed in `CLAUDE.md`.